

UCS503 – Software Engineering Project

Academic Year 2026–2027

NurseSync AI

Intelligent Audio-Based Clinical Handoff System

*Voice-Driven Handoff Capture, Speech-to-Text and AI-Assisted Clinical
Information Retrieval*

Submitted by:

1024030732 Shivansh Sharma

1024030941 Pushp Batheja

1024030950 Ishaan Bhalla

B.Tech Third Year, Computer Engineering

Submitted to:

Dr. Jeelani Asif

Department of Computer Science and Engineering

Mid-Semester Evaluation Report

Computer Science and Engineering Department
Thapar Institute of Engineering and Technology, Patiala

Contents

1	Introduction	1
1.1	Background and Context	1
1.2	Problem Statement	1
1.3	Project Objectives	1
2	Methodology and Design	3
2.1	System Architecture	3
2.1.1	High-Level Design	3
2.1.2	Technical Stack	5
2.2	Data Flow Design	5
2.2.1	Handoff Processing Pipeline	7
3	Implementation and Evaluation	8
3.1	Implementation Status	8
3.1.1	FastAPI Backend and CORS	8
3.1.2	Speech-to-Text Implementation	9
3.1.3	RAG and Vector Search Design	9
3.2	Testing Methodology	9
3.3	Current Results	10
4	Conclusion	11
4.1	Summary of Work	11
4.2	Key Findings	11
4.3	Future Work and Recommendations	11
4.4	Final Remarks	12

Chapter 1

Introduction

1.1 Background and Context

Clinical handoffs are a critical part of nursing workflow because responsibility for a patient is transferred from one caregiver or shift to another. In practice, handoff information may be communicated verbally while also being documented manually. This creates a repetitive documentation burden and can make it difficult for the incoming nurse to quickly reconstruct what changed, what remains pending, and which observations require attention.

NurseSync AI is designed as a voice-driven clinical handoff system. The central idea is to allow a nurse to speak a handoff naturally, capture the audio, convert the speech into text, and progressively transform the information into structured and searchable clinical records. The proposed system combines a Flutter-based application, a FastAPI backend, speech-to-text processing, clinical NLP/LLM components, PostgreSQL storage, and retrieval-augmented question answering.

1.2 Problem Statement

Manual clinical handoff documentation requires nurses to divide attention between patient care and data entry. A purely verbal handoff can also be difficult to search or review later, while unstructured notes can omit important fields such as medications, allergies, vitals, tests, or pending actions.

NurseSync AI addresses this problem by introducing an audio-first workflow in which the spoken handoff becomes the starting point for structured documentation. The current delivered prototype focuses on the first working AI stage: end-to-end speech-to-text. The broader architecture provides a staged path toward clinical entity extraction, SBAR structuring, timeline generation, missing-information detection, risk decision support, and RAG-based patient-specific question answering.

1.3 Project Objectives

The core objectives of NurseSync AI are:

1. **Voice-based handoff capture:** allow a nurse to record a bedside handoff without manually typing the entire interaction.
2. **Speech-to-text transcription:** convert recorded clinical speech into a readable transcript using a local Whisper-family model.
3. **Clinical structuring:** extract useful medical entities such as vitals, medications, diagnoses, allergies, and tests from the transcript in later stages.
4. **Automated handoff organization:** generate SBAR-style handoffs, concise summaries, and timestamped timeline events.
5. **Decision support:** experiment with completeness checking and deterioration-risk scoring while keeping the nurse responsible for verification.
6. **Patient-specific AI assistance:** use retrieval-augmented generation so questions can be answered from relevant patient records rather than unsupported model knowledge.

Safety boundary: NurseSync AI is intended as a documentation and decision-support system. Its summaries, retrieved answers, and experimental risk outputs are not intended to replace clinical judgment or provide autonomous diagnosis.

Chapter 2

Methodology and Design

2.1 System Architecture

2.1.1 High-Level Design

NurseSync follows a layered architecture in which the user-facing application is separated from the backend processing pipeline. The intended flow is: nurse input, mobile/web interface, FastAPI APIs, AI processing, persistent storage, and retrieval for subsequent users.

NurseSync AI — Clinical Handoff Workflow (UML Activity Diagram)

Voice handoff capture through AI processing, storage, and RAG-assisted retrieval

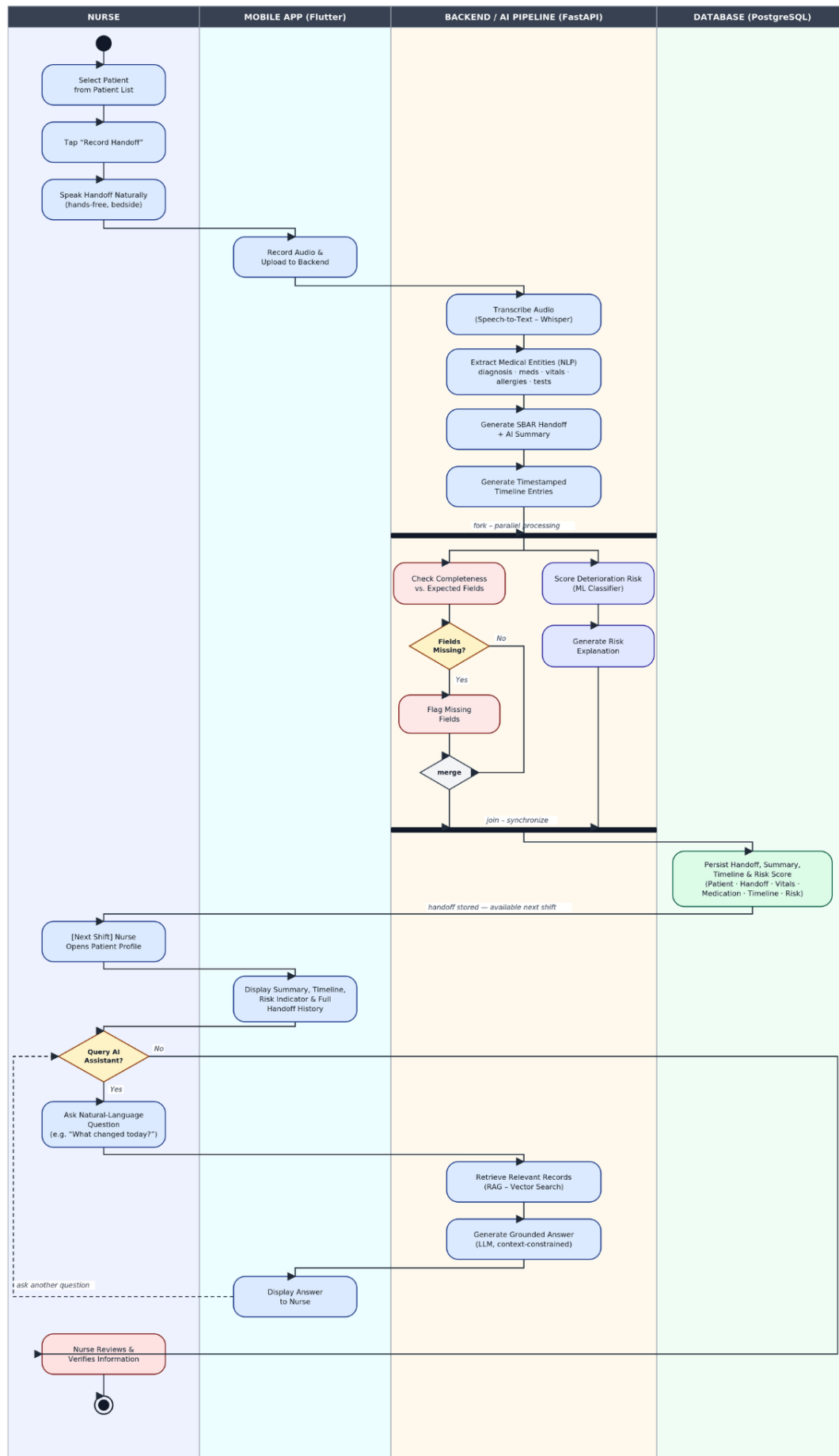


Figure 2.1: NurseSync AI Clinical Handoff Workflow – UML Activity Diagram

The activity diagram shows the complete proposed workflow. The nurse selects a patient, starts a handoff, and speaks naturally. The application records and uploads audio. The backend performs speech-to-text, medical entity extraction, SBAR and summary generation, and timeline generation. Completeness checking and deterioration-risk scoring are then designed to run in parallel before the resulting information is persisted. On a subsequent shift, the nurse can view the patient history and optionally ask a natural-language question through the RAG-assisted AI interface.

2.1.2 Technical Stack

The system uses mature components so that engineering effort can be concentrated on integration and correctness:

- **Frontend:** Flutter is the intended cross-platform application framework; the currently delivered prototype also contains a standalone speech-to-text web page used for the working demonstration.
- **Backend:** FastAPI provides the REST API layer and coordinates audio processing.
- **Speech recognition:** faster-whisper provides local speech-to-text inference. The current implementation runs the model locally on CPU using integer quantisation to reduce resource requirements.
- **Data storage:** PostgreSQL is the planned relational store for patients, handoffs, vitals, medications, timelines, and risk information.
- **RAG layer:** embeddings and vector search are planned for retrieving semantically relevant patient records before LLM answer generation.
- **Machine learning:** scikit-learn/XGBoost-style experiments are planned for deterioration-risk classification and explainability.

2.2 Data Flow Design

The Data Flow Diagrams divide the system into external actors, processes, data stores, and data flows. Level 0 treats NurseSync AI as one system and shows its interaction with the nurse, administrator, and AI/ML services. Level 1 decomposes the system into handoff capture, handoff processing, data storage, AI assistant, and risk processing. Level 2 expands the handoff-processing stage into transcription, NLP extraction, SBAR/summary generation, timeline generation, risk prediction, and missing-information checking.

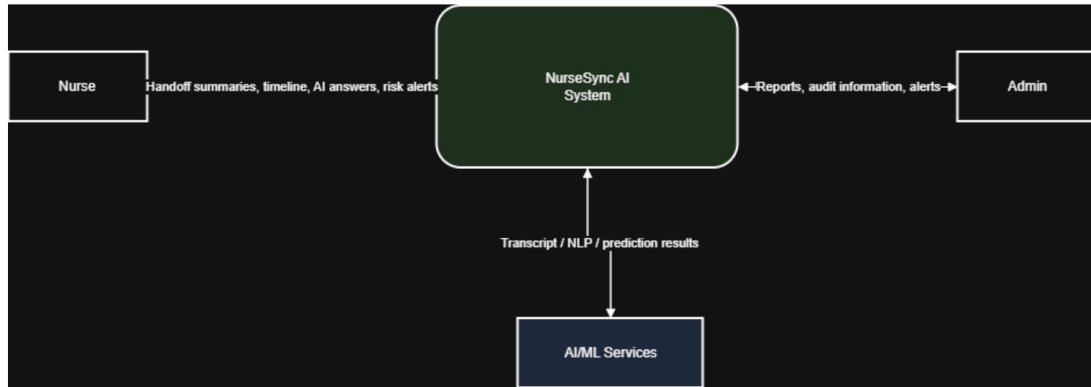


Figure 2.2: NurseSync AI – Level 0 Context Data Flow Diagram

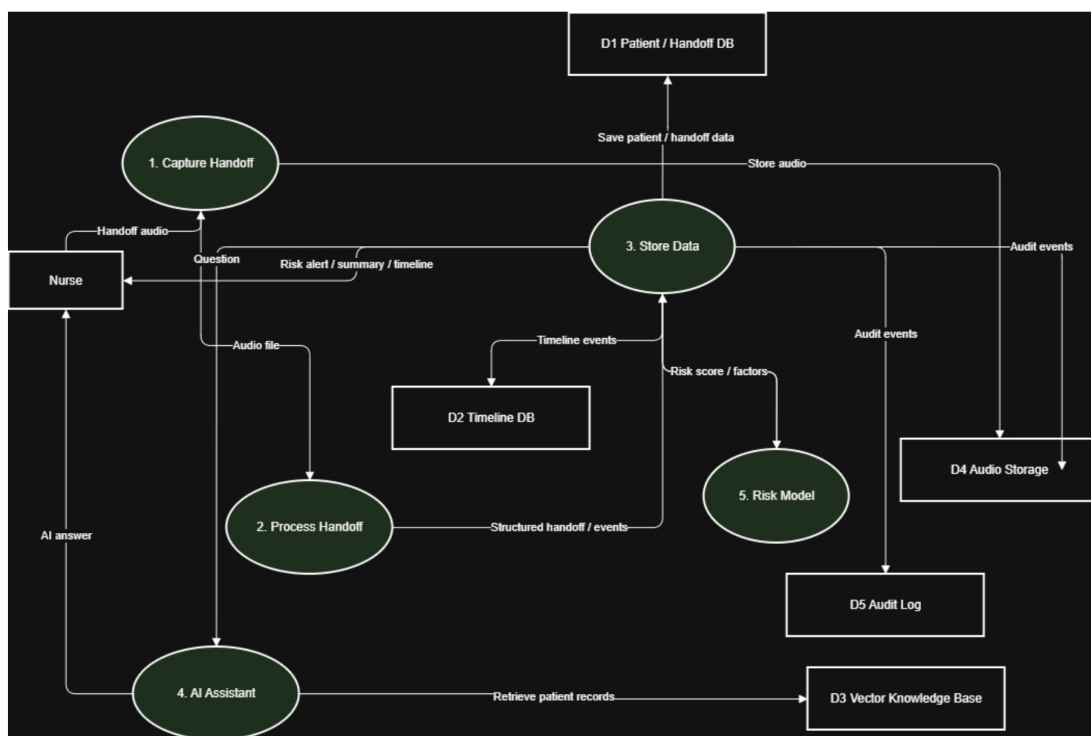


Figure 2.3: NurseSync AI – Level 1 Data Flow Diagram

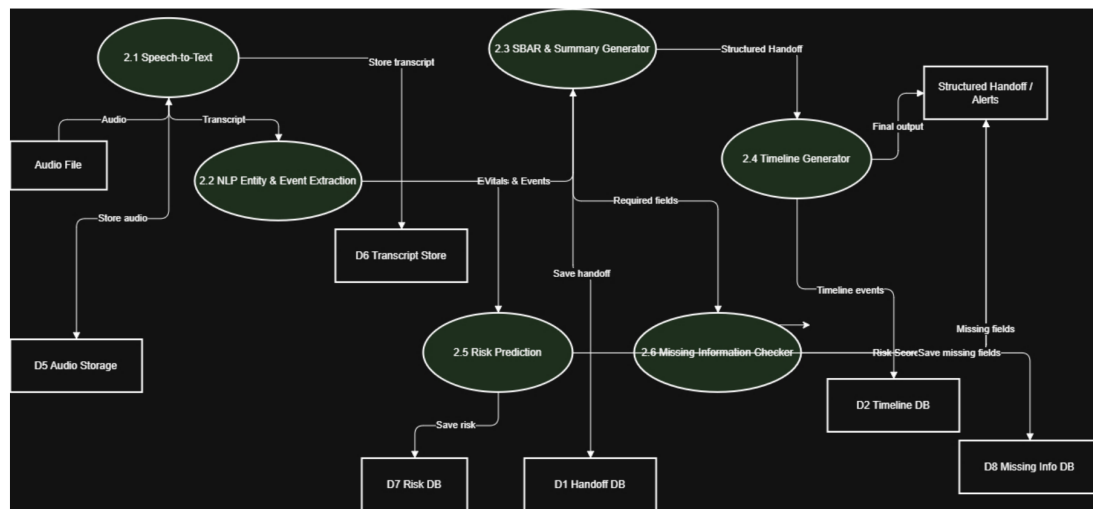


Figure 2.4: NurseSync AI – Level 2 Handoff Processing Data Flow Diagram

2.2.1 Handoff Processing Pipeline

The detailed processing pipeline is designed as:

1. **Audio input:** a nurse records a spoken handoff.
2. **Speech-to-text:** the audio is passed to faster-whisper and converted into a transcript.
3. **Medical entity extraction:** an NLP/LLM stage identifies clinically meaningful information.
4. **SBAR and summary generation:** extracted information is organized into Situation, Background, Assessment, and Recommendation fields and a concise summary.
5. **Timeline generation:** clinically relevant events are converted into timestamped timeline entries.
6. **Completeness checking:** expected fields are compared with the extracted information and missing items can be flagged.
7. **Risk prediction:** an experimental classifier produces a deterioration-risk score and an explanation of contributing factors.
8. **Persistence:** structured handoff information, summaries, timeline events, and risk information are stored for later retrieval.

Chapter 3

Implementation and Evaluation

3.1 Implementation Status

The project is being delivered incrementally. The current repository contains a working speech-to-text demonstration rather than the full proposed clinical application. The implemented work includes the FastAPI application setup, local faster-whisper integration, the transcription endpoint, a speech-to-text frontend page, and end-to-end verification from recording through transcription and display.

The broader PostgreSQL-backed clinical data model, RAG assistant, completeness checker, and risk model remain part of the staged development roadmap. They are therefore described in this report as planned or architectural components rather than as completed production functionality.

3.1.1 FastAPI Backend and CORS

FastAPI acts as the server-side entry point. The backend exposes an HTTP endpoint that accepts uploaded audio, passes the file to the transcription model, and returns a formatted transcript to the frontend.

For browser-based development, Cross-Origin Resource Sharing (CORS) is configured using FastAPI's `CORSMiddleware`. CORS allows the backend to explicitly control which frontend origins, methods, headers, and credentials may be used for cross-origin requests. A permissive configuration can be used during local development, while a production deployment should restrict the allowed origins to trusted clients.

A simplified configuration is:

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
```

```
allow_methods=["*"],  
allow_headers=["*"],  
)
```

The important implementation idea is that CORS is a communication/security mechanism, not an AI component. It allows a browser-based client to call the FastAPI service when the request origin satisfies the configured policy.

3.1.2 Speech-to-Text Implementation

The speech-to-text backend uses the faster-whisper implementation of Whisper for local inference. The model is deliberately run locally so that the prototype does not require a hosted transcription API key or a per-request transcription service. Once the model is available locally, transcription can also be performed without sending the recorded audio to an external transcription provider.

The end-to-end flow is:

Record Audio → Upload → FastAPI → faster-whisper → Transcript → Display

The frontend exposes clear recording states so that the user can distinguish between recording and transcription. The current speech-to-text page captures microphone audio, uploads the resulting recording, and displays the returned transcript.

3.1.3 RAG and Vector Search Design

The proposed AI assistant uses Retrieval-Augmented Generation (RAG). When a nurse asks a question such as “What changed today?”, the system first retrieves relevant patient records through vector search. The retrieved information is then supplied to an LLM as context, allowing the generated answer to remain grounded in the patient’s available records.

Text or record chunks can be transformed into numerical embeddings. Vector search compares the embedding of the nurse’s question with stored embeddings and returns semantically relevant records. This differs from a simple keyword search because related concepts such as “oxygen level” and “SpO2” can be represented as semantically similar information.

3.2 Testing Methodology

Testing of the current delivered prototype focuses on the speech-to-text path. The end-to-end test verifies the complete chain from microphone recording to transcription and display. Backend testing also checks that the FastAPI service can receive the uploaded audio and return a usable response.

The planned evaluation framework expands this into multiple measurable dimensions:

- **Handoff Documentation Time:** time from recording start to a fully structured and stored handoff.
- **Entity extraction accuracy:** precision, recall, and F1 for fields such as diagnosis, medication, vitals, allergies, and tests.
- **Completeness detection:** precision and recall for intentionally omitted handoff fields.
- **RAG groundedness:** proportion of assistant answers fully supported by retrieved patient records.
- **Risk model performance:** accuracy, precision, recall, F1, and ROC-AUC, with particular attention to sensitivity and false negatives.
- **Usability:** nurse feedback on summary, timeline, and AI-assistant usefulness.

3.3 Current Results

The current prototype demonstrates the core technical feasibility of the first AI stage: a recorded voice handoff can be sent to the FastAPI backend, processed locally with faster-whisper, and returned as text for display. This validates the basic audio-to-transcript communication path required for later clinical structuring stages.

Current implementation boundary: the report intentionally distinguishes implemented functionality from proposed architecture. The current delivered repository is centered on the standalone speech-to-text demonstration; the full PostgreSQL clinical workflow, RAG assistant, and deterioration-risk pipeline are subsequent development stages.

Chapter 4

Conclusion

4.1 Summary of Work

NurseSync AI establishes an engineering foundation for voice-driven clinical handoff documentation. The project models the workflow using UML activity and data-flow diagrams and defines a staged architecture connecting a user interface, FastAPI backend, speech recognition, clinical AI processing, relational storage, and retrieval-assisted question answering.

The first working implementation focuses on speech-to-text. The team has set up the FastAPI service, integrated faster-whisper for local transcription, implemented the transcription endpoint and a frontend recording page, and verified the end-to-end recording-to-transcript flow.

4.2 Key Findings

1. Local faster-whisper transcription provides a practical first AI component without requiring a hosted transcription API.
2. Separating the frontend from the FastAPI backend gives the project a clean boundary for later AI and database integration.
3. The DFD and activity-diagram decomposition makes the planned clinical pipeline explicit and provides a roadmap for incremental implementation.
4. RAG can provide a safer patient-specific question-answering architecture by retrieving relevant records before asking an LLM to generate an answer.
5. Human verification remains essential because transcription, extraction, summarization, retrieval, and risk models can all produce errors.

4.3 Future Work and Recommendations

The next development stages are:

1. Integrate the patient and handoff data model with PostgreSQL.
2. Add medical entity extraction and structured clinical fields.

3. Generate SBAR handoffs, concise summaries, and timestamped timelines.
4. Implement the missing-information checker and review/correction workflow.
5. Build the RAG assistant with embeddings and vector retrieval over patient records.
6. Develop and evaluate the experimental deterioration-risk classifier with explainability.
7. Add stronger authentication, access control, audit logging, automated tests, CI/CD, and secure deployment before any real clinical use.

4.4 Final Remarks

NurseSync AI is designed to reduce documentation friction while improving the accessibility and continuity of clinical handoff information. The current speech-to-text prototype provides a concrete first increment, while the architecture and design artifacts establish a path toward structured handoffs, searchable timelines, grounded AI assistance, and explainable decision support. Any future clinical deployment must use appropriate privacy safeguards, synthetic or de-identified development data, human verification, and rigorous evaluation before use with real patient information.

Bibliography

- [1] A. Radford et al., “Robust Speech Recognition via Large-Scale Weak Supervision,” OpenAI, 2022.
- [2] FastAPI Documentation, “FastAPI – Modern, fast web framework for building APIs with Python.”
- [3] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *Advances in Neural Information Processing Systems*, 2020.
- [4] PostgreSQL Global Development Group, “PostgreSQL Documentation.”
- [5] Flutter Documentation, “Build apps for any screen.”
- [6] NurseSync AI Team, “NurseSync AI – Intelligent Audio-Based Clinical Handoff System,” Project Proposal, Thapar Institute of Engineering and Technology, 2026.